DG INTERNS HUB

Cybersecurity Internship Program

WEEK 9 ASSIGNMENT REPORT
SOC Introduction & Setup — Snort IDS Lab

Prepared by

Prakhar Gupta

Submission Date

5/11/2026

Environment / Tools

Kali Linux (VMware Workstation), Snort IDS (Snort++ 3.12.2.0)

Kali Linux (VMware Workstation), Snort IDS (Snort++ 3.12.2.0)

Table of Contents

1. Introduction 3
1.1 Scope of This Week's Assignment 3
1.2 A Note on the Environment Actually Used 3
2. SOC Fundamentals 3
2.1 What Is a Security Operations Center? 3
2.2 Analyst Roles: L1, L2 and L3 4
2.3 SIEM vs. IDS vs. IPS 4
2.4 Where Detection Engineering Fits 4
3. Environment Setup 5
3.1 Host Identification 5
3.2 Network Configuration 5
3.3 Package Index Refresh 6
4. Snort IDS Installation 6
4.1 Verifying the Installation 7
4.2 Configuration Syntax Validation 7
5. Custom Detection Rules 9
5.1 Anatomy of a Snort Rule 9
5.2 Rule 1 — ICMP Ping Detection 9
5.3 Rule 2 — HTTP Traffic Detection 9
5.4 Negative Test: Confirming the Validation Safety Net 10
6. Testing & Validation 10
6.1 ICMP Rule Test 10
6.2 HTTP Rule Test 13
6.3 Observations 14
7. Results Summary 14
8. Appendix A — Custom Rule File 14
9. Remediation & Recommendations for Production Use 16
10. Conclusion 16

1. Introduction

This report documents Week 9 of the DG Interns Hub Cyber Security Internship, which shifts focus from offensive testing (covered in the Week 8 VAPT exercise against OWASP Juice Shop) to the defensive side of security operations. The objective was twofold: first, to build a conceptual understanding of how a Security Operations Center (SOC) functions, including analyst roles and the distinction between SIEM, IDS and IPS technologies; and second, to put that theory into practice by standing up a minimal, self-hosted monitoring environment using a Linux host and the Snort intrusion detection engine.

All setup and testing described below was carried out entirely inside an isolated virtual machine (Kali Linux, running under VMware Workstation) using free and open-source tools — Snort 3 ("Snort++") and its Community rule syntax — with no production systems, external networks, or third-party infrastructure involved at any stage beyond ordinary outbound package downloads and a handful of ICMP/HTTP requests to public internet hosts used purely as test traffic.

1.1 Scope of This Week's Assignment

- Task 1 — Research and document SOC fundamentals: definition, analyst tiers (L1/L2/L3), and SIEM vs. IDS vs. IPS.
- Task 2 — Provision/confirm a Linux host inside a virtual machine and perform basic system/network setup.
- Task 3 — Install Snort IDS on the host and verify the installation and configuration.
- Task 4 — Author two custom Snort detection rules (ICMP ping detection and HTTP traffic detection).
- Task 5 — Generate matching traffic and capture live Snort alerts as evidence that the rules fire correctly.

1.2 A Note on the Environment Actually Used

The assignment brief suggested a freshly-provisioned Ubuntu Server as the lab host. In practice, this exercise was carried out on an existing Kali Linux virtual machine (VMware Workstation) that was already available in the internship lab environment. Kali is itself a Debian-derived distribution built for security tooling, ships Snort directly in its package repositories, and required no additional base-OS installation step — so Section 3 below documents confirming and configuring that existing host rather than installing a new one from an ISO. This substitution does not change anything about how Snort itself was installed, configured or tested.

2. SOC Fundamentals

Task 1 — SOC Basics

2.1 What Is a Security Operations Center?

A Security Operations Center (SOC) is the centralized function — people, processes and technology — responsible for continuously monitoring, detecting, investigating and responding to cybersecurity threats across an organization's networks, systems and endpoints. Rather than being a single tool, a SOC is best understood as an operating model: it ingests telemetry from across the environment (logs, network traffic, endpoint activity, cloud events), correlates that telemetry against known attack patterns and threat intelligence, and routes anything suspicious through a defined triage and escalation workflow so that genuine incidents are contained before they cause material damage.

A mature SOC typically operates on a 24x7 basis and is measured on a small set of operational metrics: mean time to detect (MTTD), mean time to respond (MTTR), and the false-positive rate of its detection content. Reducing all three simultaneously — catching real attacks faster while not drowning analysts in irrelevant alerts — is the central engineering challenge of running a SOC, and is the reason tuning detection rules (as practiced later in this report with Snort) is treated as a core, ongoing SOC activity rather than a one-off setup task.

2.2 Analyst Roles: L1, L2 and L3

SOC analyst work is conventionally organized into three tiers, each with a different scope of responsibility and escalation path:

- L1 Analyst (Triage): The first responder to alerts generated by SIEM/IDS tooling. Responsible for initial validation — confirming whether an alert reflects real activity, gathering basic context (source/destination, user, asset), and classifying it as a false positive, a benign true positive, or something requiring deeper investigation. L1 analysts escalate anything inconclusive or high-severity to L2.
- L2 Analyst (Investigation): Performs deeper root-cause analysis on escalated alerts — correlating multiple log sources, examining endpoint or network forensic artifacts, and determining the scope and likely intent of an incident. L2 analysts typically own active incident response during business-impacting events and refine detection content based on what they find.
- L3 Analyst (Threat Hunting & Engineering): The most senior tier, focused on proactive threat hunting, malware/forensic analysis, and engineering new detection rules or SIEM use cases based on emerging threat intelligence. L3 analysts also handle the most severe or novel incidents that L2 cannot fully resolve, and own the tuning of platforms such as the IDS layer built later in this report.

2.3 SIEM vs. IDS vs. IPS

These three terms are often used loosely, but they describe distinct layers of a defensive stack:

Technology	What It Does	Where It Sits
SIEM (Security Information & Event Management)	Aggregates, normalizes and correlates logs from many sources (firewalls, servers, endpoints, IDS, cloud) to surface alerts and support investigation and compliance reporting.	Centralized — sits above the whole environment, consuming data rather than sitting inline on traffic.
IDS (Intrusion Detection System)	Passively inspects network traffic (or host activity) against signatures/rules and heuristics, and raises an alert when a match occurs. It observes and reports but does not itself block traffic.	Out-of-band (e.g. a mirrored port) or inline in “detect-only” mode — the mode Snort is run in for this exercise.
IPS (Intrusion Prevention System)	Uses the same signature/heuristic matching as an IDS, but sits inline on the traffic path and can actively drop or reject packets that match a malicious rule, stopping the attack in real time rather than just logging it.	Inline, in the direct path of traffic — every packet passes through it before reaching its destination.

Table 1: Core differences between SIEM, IDS and IPS.

In short: a SIEM is a visibility and correlation layer sitting above everything else; an IDS is a sensor that watches traffic and raises alarms; and an IPS is the same sensor technology given the authority to act on what it sees. Snort itself illustrates this spectrum well — it ships as a single engine that can run in either IDS mode (alert-only, as used throughout this report) or IPS mode (inline, with drop rules enabled).

2.4 Where Detection Engineering Fits

The rule-writing work in Sections 5–6 of this report is a small-scale version of a process real SOCs call detection engineering: an analyst (usually L2/L3) turns a hypothesis about attacker behaviour — “an attacker sweeping for live hosts sends ICMP Echo Requests”, “a web request to our stack looks like plaintext HTTP on port 80” — into a concrete, testable rule, validates that the rule's syntax and logic are correct, deploys it, and then confirms against live or simulated traffic that it fires on the intended behaviour and stays quiet otherwise. That four-step loop (hypothesis → rule → validation → live test) is exactly the structure followed in Tasks 4 and 5 below.

3. Environment Setup

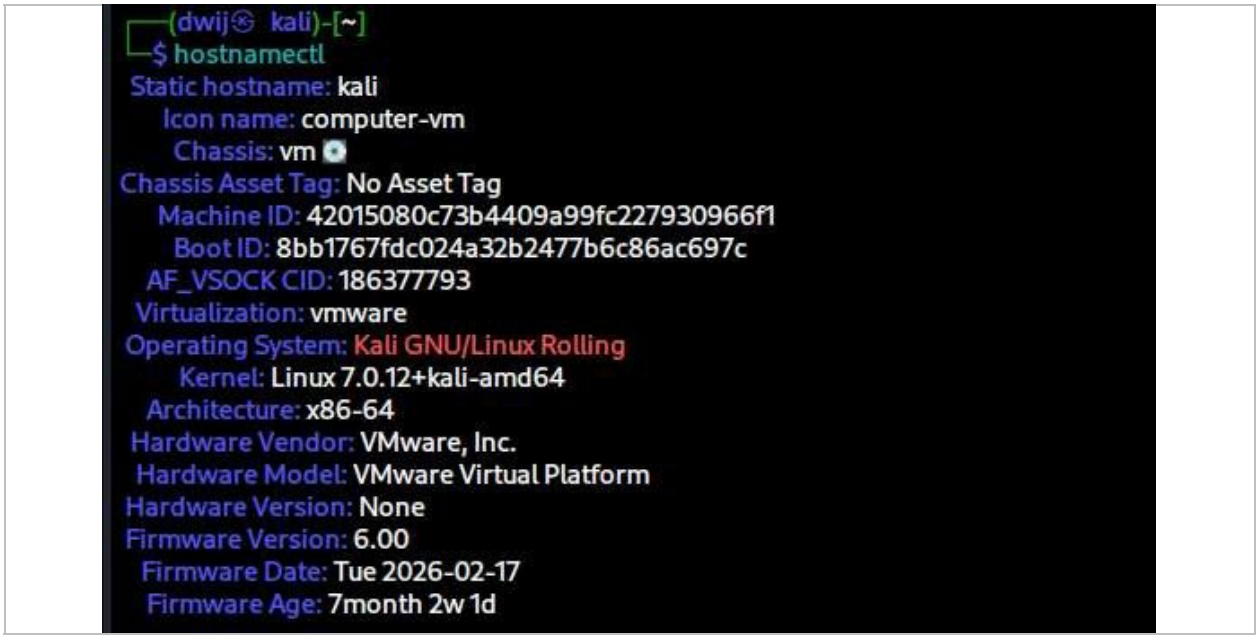
Task 2 — Host & Network Configuration

As noted in Section 1.2, the lab host used for this exercise was an existing Kali Linux virtual machine running under VMware Workstation, rather than a freshly-installed Ubuntu Server. The steps below confirm and record that host's identity and network configuration before Snort is installed on it, so that later evidence (Sections 4–6) can be tied back to a known, documented sensor.

3.1 Host Identification

The host's identity was confirmed with hostnamectl, which reports the static hostname together with OS, kernel and virtualization details:

hostnamectl

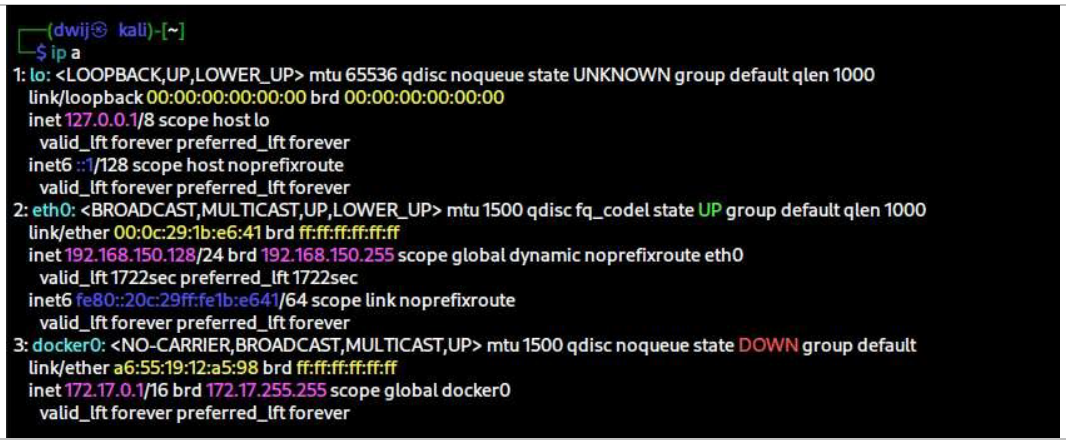


Screenshot 1a: hostnamectl output — confirms the lab host is a Kali GNU/Linux Rolling VM (hostname “kali”), kernel 7.0.12+kaliamd64, running under VMware virtualization.

3.2 Network Configuration

The assigned network interface and IP address were identified with ip a, which is needed later to tell Snort which interface to bind to:

```
ip a
```

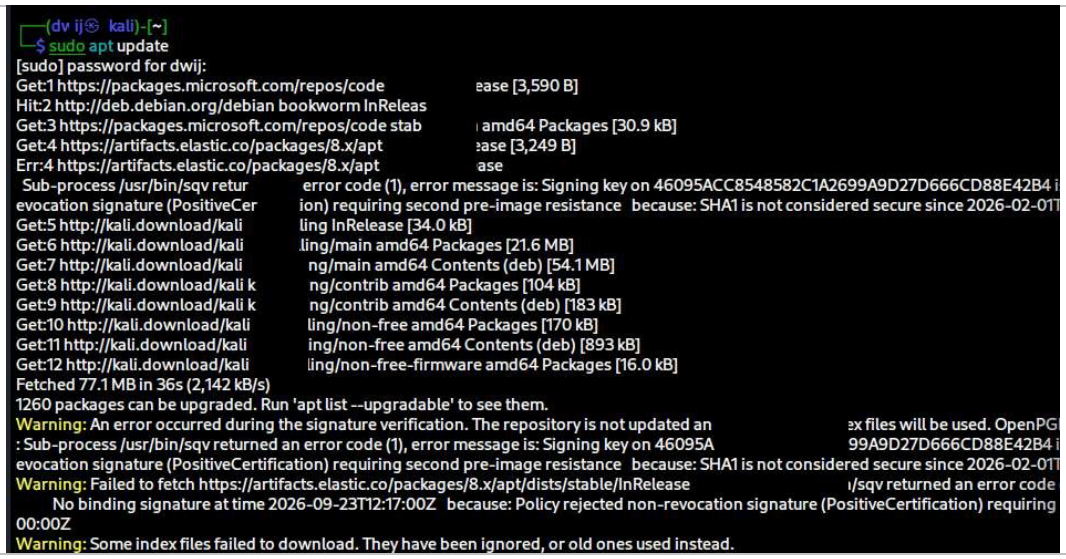


Screenshot 1b: ip a output — the sensor's primary interface eth0 is assigned 192.168.150.128/24; loopback (lo) and an unused docker0 bridge are also present but not used for this exercise.

3.3 Package Index Refresh

The package index was refreshed before installing Snort and its dependencies, so the install would pull current package versions:

```
sudo apt update
```



Screenshot 1c: sudo apt update output — the refresh completed and fetched updated package lists; several repository signing-key warnings were logged for a third-party (Elastic) repository unrelated to Snort and did not block the refresh or any later step.

With the host identified, its network interface confirmed, and the package index refreshed, the host was ready for the Snort installation covered in the next section.

4. Snort IDS Installation

Task 3 — Installing and Verifying Snort

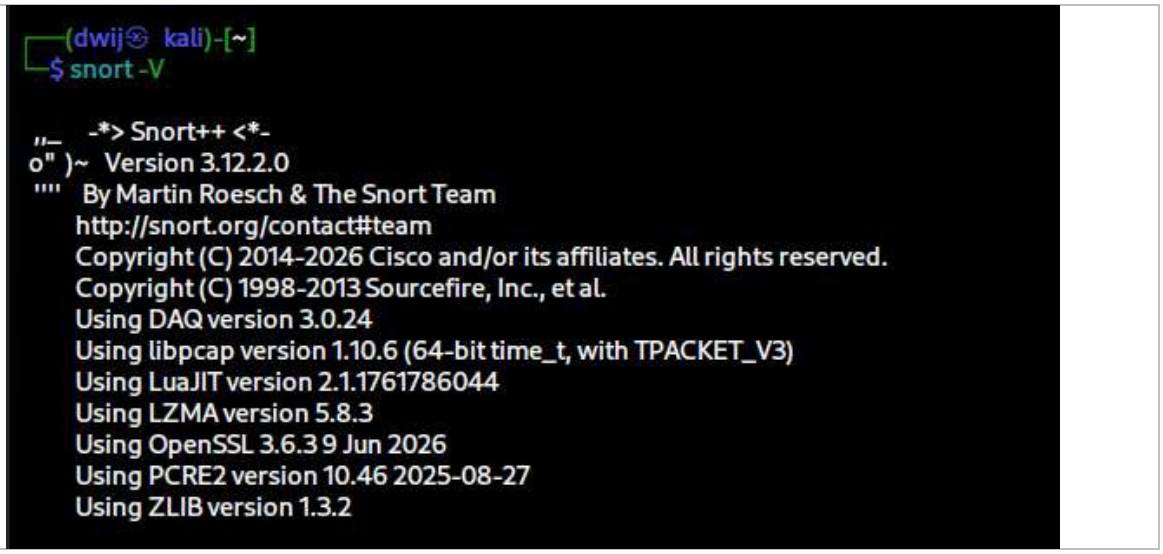
Snort was chosen for this exercise because it is free, open-source, and the de facto reference implementation that most commercial and open SIEM/IDS stacks (including Suricata-based ones) trace their rule syntax back to — making it the most transferable tool to learn the underlying detection logic on. Kali Linux carries Snort 3 (“Snort++”) directly in its repositories, so installation was a single package command rather than a build from source:

```
sudo apt install -y snort
```

4.1 Verifying the Installation

After installation, the Snort binary version and build configuration were checked to confirm a working install before any rules were touched:

```
snort -V
```



Screenshot 2: Output of snort -V, showing the installed Snort++ 3.12.2.0 build together with its DAQ, libpcap, LuaJIT, OpenSSL, PCRE2 and ZLIB library versions — confirms the binary installed and linked correctly.

4.2 Configuration Syntax Validation

Before running Snort against live traffic, its configuration and rule set were validated in test mode (-T), which parses everything and exits without starting a capture. The first attempt used the classic Snort 2-style path, /etc/snort/snort.conf, out of habit from older documentation — this failed immediately, because Snort 3 replaces the old .conf format with a Lua-based configuration file:

```
sudo snort -T -c /etc/snort/snort.conf -i eth0
```



Screenshot 3a: First validation attempt against the legacy /etc/snort/snort.conf path — fails with “cannot open /etc/snort/snort.conf: No such file or directory”, since Snort 3 does not use this file.

The command was corrected to point at the actual Snort 3 configuration file, /etc/snort/snort.lua, which loaded successfully and walked through every inspector module (stream, http_inspect, appid, and others) before confirming a clean parse:

```
sudo snort -c /etc/snort/snort.lua -T -i eth0
```



Screenshot 3b: Corrected validation run against /etc/snort/snort.lua — every inspector module loads cleanly in sequence, confirming the base configuration itself is valid.

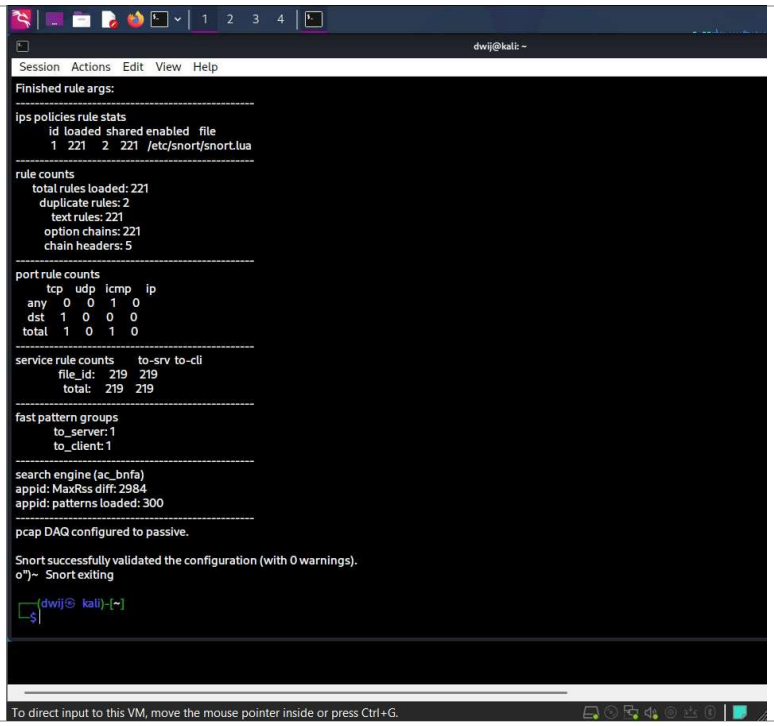
With the base configuration confirmed, the same test was repeated while explicitly pointing Snort at the custom rule file that Task 4 populates (Section 5), to confirm the rule file's own syntax independently of the base config:

```
sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -T
```



Screenshot 3c: Validation run including -R /etc/snort/rules/local.rules — the rule file loads through the same inspector sequence without errors.

The final summary line confirmed a fully clean validation of both the base configuration and the custom rule file together:



Screenshot 3d: Final validation summary — port/rule counts show 1 ICMP rule and 1 TCP rule loaded (exactly the two custom rules from Task 4), and Snort reports "successfully validated the configuration (with 0 warnings)" before exiting.

5. Custom Detection Rules

Task 4 — Writing Snort Rules

5.1 Anatomy of a Snort Rule

Every Snort rule is a single line split into two parts: a rule header (action, protocol, source/destination IP and port) and a set of options in parentheses (match conditions, alert message and a unique signature ID). The general shape is:

```
action protocol src_ip src_port -> dst_ip dst_port (option1; option2; ... ; sid:NNNNNN; rev:1;)
```

Two fields matter most for building readable, maintainable rules: msg, which sets the human-readable text shown in the alert console, and sid, a locally-unique numeric rule ID — Snort reserves 1,000,000 and above for user-defined rules, which is the range used for both custom rules below. Both rules were written into /etc/snort/rules/local.rules, the conventional location for site-specific custom rules, kept separate from the vendor-maintained Community rule set.

5.2 Rule 1 — ICMP Ping Detection

This rule flags any ICMP Echo Request (a standard ping), which is a common and simple reconnaissance technique an attacker uses to sweep a subnet for live hosts before attempting anything more targeted:

```
alert icmp any any -> $HOME_NET any (msg:"SOC-ALERT ICMP Ping Detected"; itype:8; sid:1000001; rev:1;)
```

itype:8 restricts the match specifically to Echo Request packets (as opposed to Echo Reply or other ICMP message types), so the rule fires only on pings rather than on every ICMP packet that crosses the sensor.

5.3 Rule 2 — HTTP Traffic Detection

This rule flags TCP connections to port 80, giving basic visibility into plaintext web traffic touching the lab network:

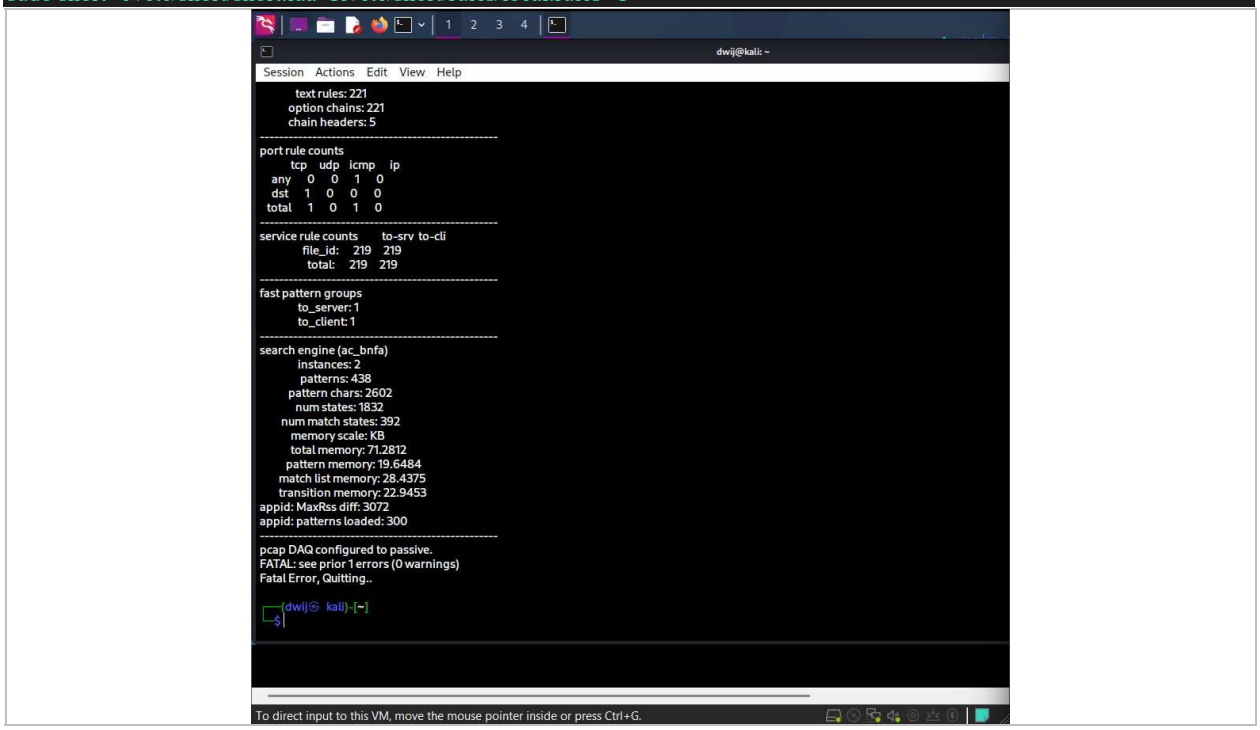
```
alert tcp any any -> $HOME_NET 80 (msg:"SOC-ALERT Inbound HTTP Connection"; sid:1000002; rev:1;)
```

A practical note from Section 6: on this single-interface lab sensor, HOME_NET was left at Snort's default value (any) rather than being narrowed to a specific subnet. That default meant both rules matched traffic regardless of which direction it crossed the monitored interface in — which is why Section 6 shows the ICMP rule firing on pings the sensor itself sent out as well as pings aimed at it, and the HTTP rule firing on an outbound web request from the sensor rather than only on inbound connections. For a production deployment, HOME_NET would normally be scoped to the protected subnet specifically so that only genuinely inbound traffic is treated as reconnaissance (see Section 9).

5.4 Negative Test: Confirming the Validation Safety Net

To confirm that the -T validation step (Section 4.2) would actually catch a broken rule before it ever reached a live sensor, a deliberate syntax error was introduced into local.rules and the same validation command was re-run:

```
sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -T
```



Screenshot 3e: Validation run against a deliberately broken rule file — Snort reports “FATAL: see prior 1 errors (0 warnings)” and exits without starting, instead of silently loading a malformed rule.

This confirmed that a malformed rule is caught at validation time with a clear, fatal error rather than being silently ignored or crashing a live capture session — an important safety property for any sensor that is updated routinely. The error was then reverted before continuing to Task 5.

6. Testing & Validation

Task 5 — Generating Traffic and Capturing Alerts

With both custom rules in place and validated, Snort was started in live mode, printing fast-style alerts straight to the console as matching traffic arrived.

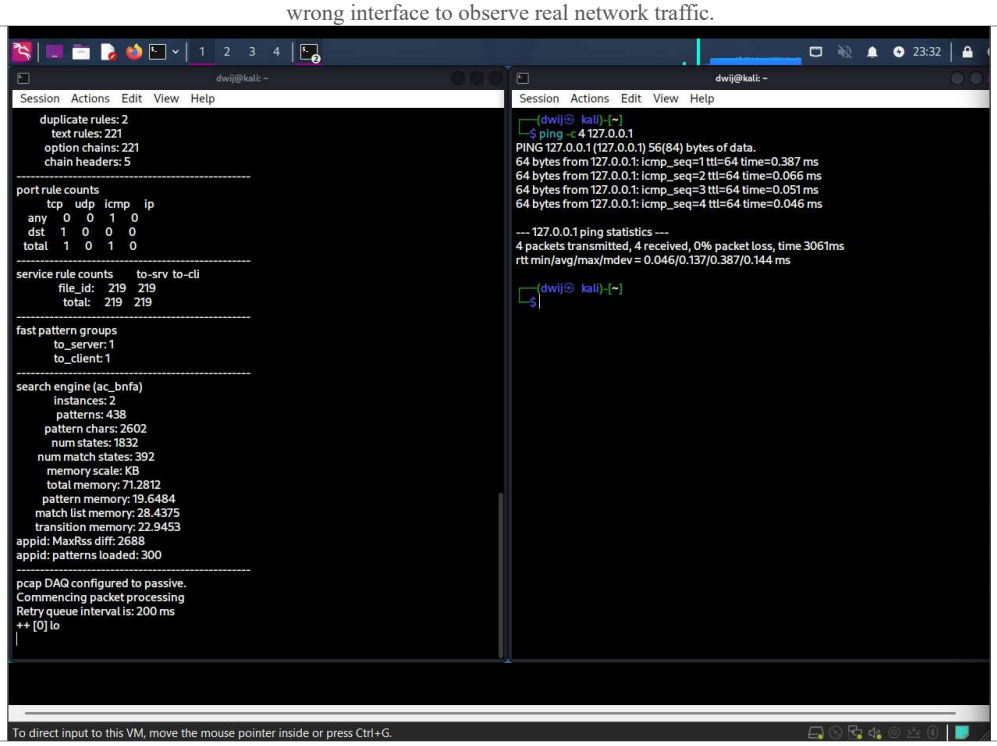
6.1 ICMP Rule Test

The first live run bound Snort to the loopback interface (lo) by default rather than the real network adapter. Test traffic was sent as a loopback ping, which Snort could see but which does not reflect genuine network reconnaissance:

```
sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -i lo ping -c 4 127.0.0.1
```

```
search engine (ac_bnfa)
           instances: 2
            patterns: 438
        pattern chars: 2602
          num states: 1832
      num match states: 392
         memory scale: KB
        total memory: 71.2812
       pattern memory: 19.6484
     match list memory: 28.4375
     transition memory: 22.9453
appid: MaxRss diff: 2720
appid: patterns loaded: 300
--------------------------------------------------
pcap DAQ configured to passive.
Commencing packet processing
Retry queue interval is: 200 ms
++ [0] lo
```

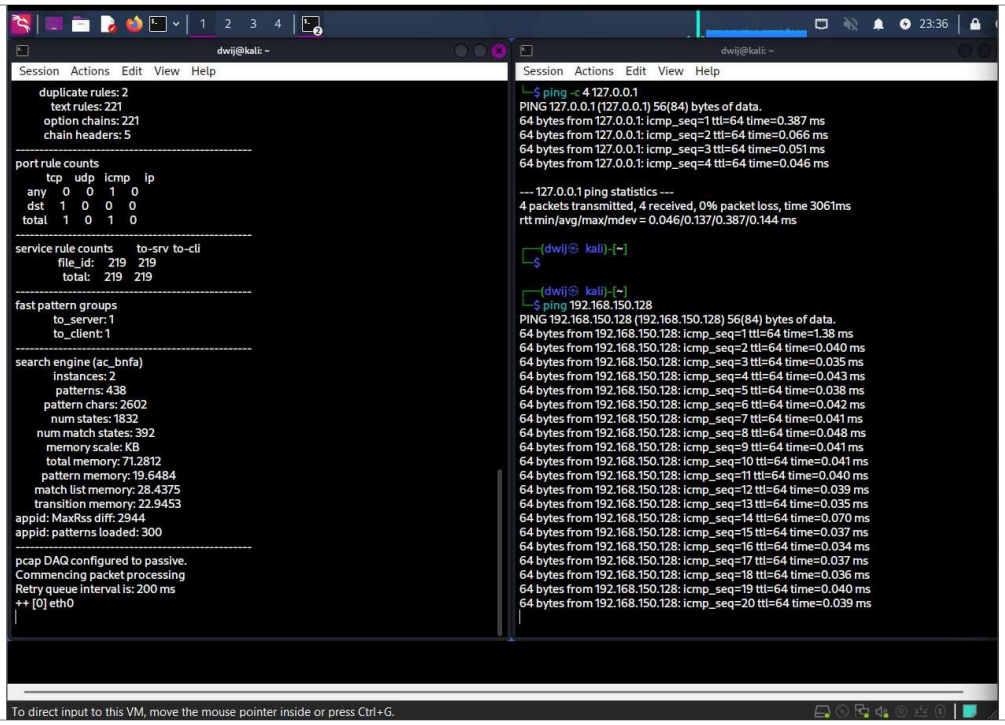
Screenshot 4a: Snort bound to the loopback interface (++ [0] lo) and waiting — a useful first check that the engine starts cleanly, but the wrong interface to observe real network traffic.



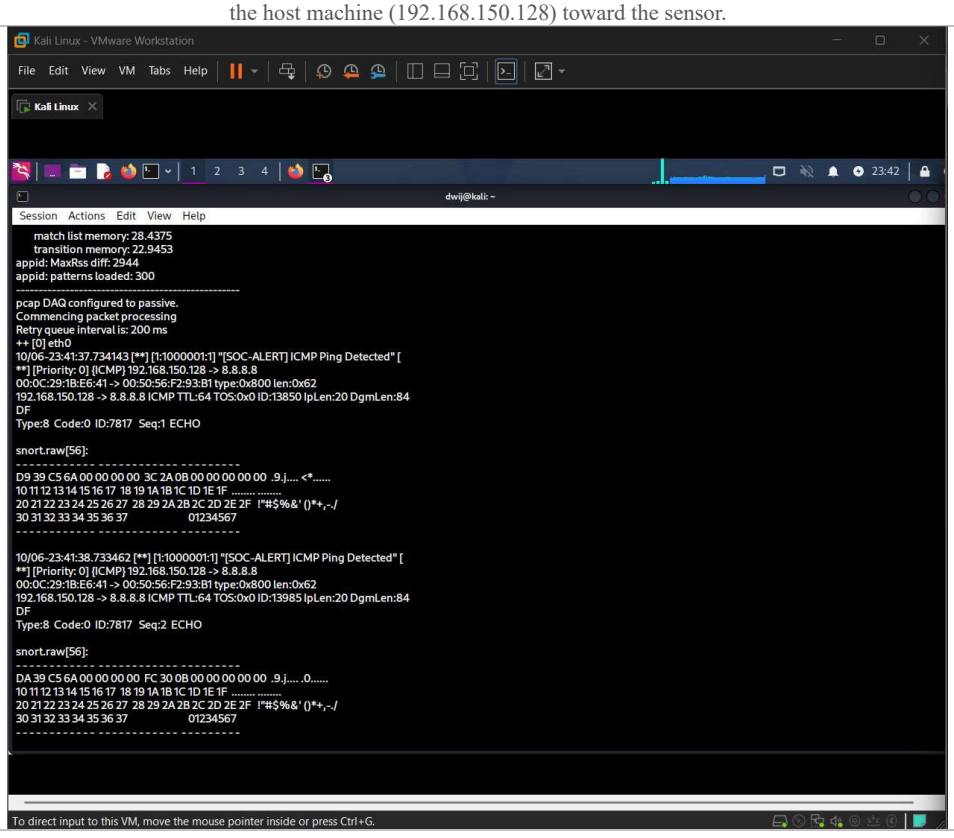
Screenshot 4b: Loopback ping test (ping -c 4 127.0.0.1) run alongside the Snort session above — confirms ICMP traffic was actually generated, as a baseline before correcting the capture interface.

Snort was then restarted bound to the real interface, eth0, identified earlier in Section 3.2, and test traffic was generated from two directions: the sensor pinging an external host, and the sensor's own host machine pinging the sensor:

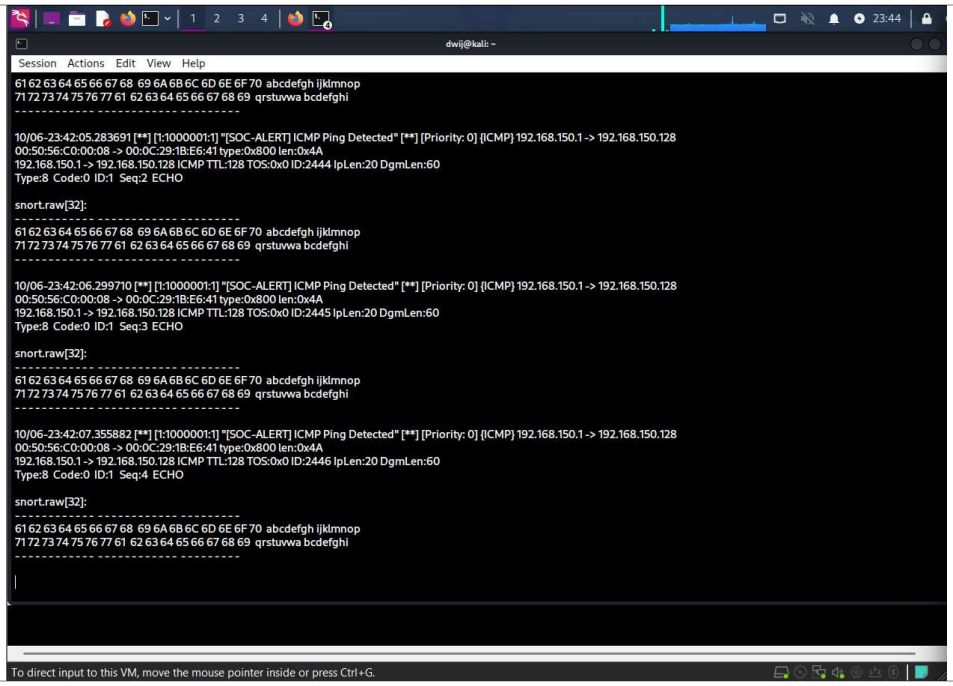
```
sudo snort -c /etc/snort/snort.lua -R /etc/snort/rules/local.rules -i eth0 ping -c 4 127.0.0.1    # baseline
ping 192.168.150.128   # from the host machine, targeting the sensor
```



Screenshot 4c: Snort bound to eth0 (++ [0] eth0) alongside the corrected ping tests — a loopback baseline and a continuous ping from the host machine (192.168.150.128) toward the sensor.



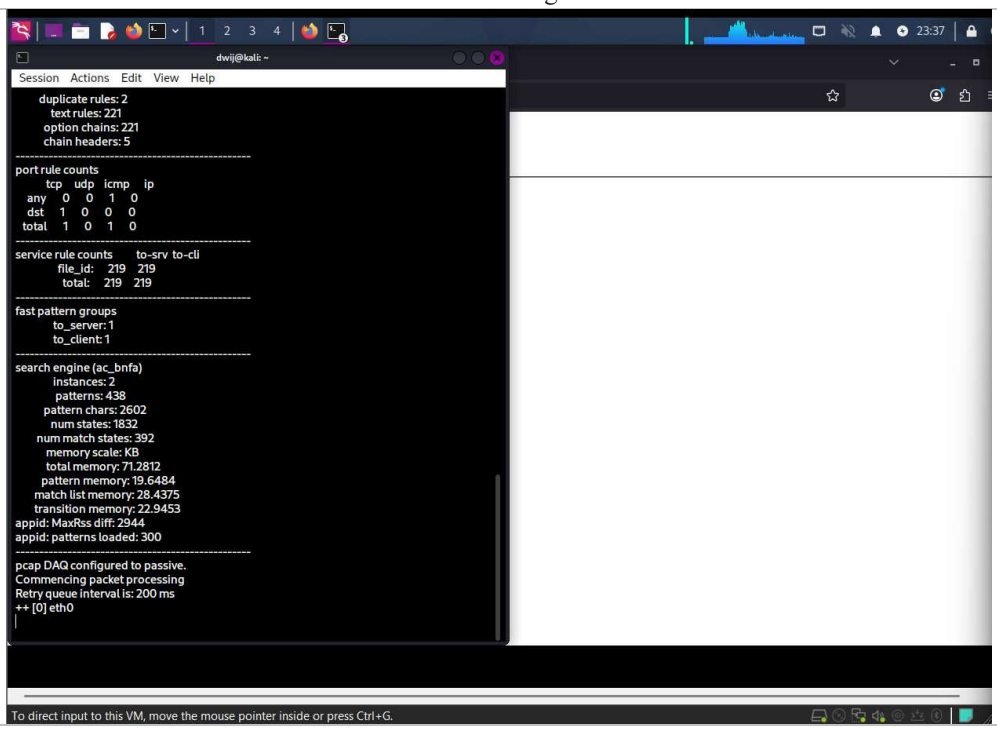
Screenshot 4d: Live Snort console on eth0 firing [SOC-ALERT] ICMP Ping Detected for outbound Echo Requests from the sensor (192.168.150.128) to 8.8.8.8, one alert per ping, each captured with its matching raw packet dump.



Screenshot 4e: The same rule firing in the opposite direction — [SOC-ALERT] ICMP Ping Detected for inbound Echo Requests from the host machine (192.168.150.1) to the sensor (192.168.150.128).

6.2 HTTP Rule Test

A browser session on the sensor's host network was used to generate an ordinary outbound web request on port 80 while the live Snort session from Section 6.1 continued running on eth0:



Screenshot 5a: Browser session prepared alongside the live Snort console on eth0, immediately before the HTTP request that triggers the rule below.



Screenshot 5b: Live Snort console firing [SOC-ALERT] Inbound HTTP Connection for the resulting TCP session from the sensor (192.168.150.128:44836) to a remote host on port 80 (172.66.147.243:80).

6.3 Observations

- The loopback-interface mistake in the first ICMP attempt (Screenshots 4a–4b) is left in the report deliberately: it is a realistic first mistake, and correcting it (binding to eth0 instead) is itself part of demonstrating correct sensor deployment, not just correct rule syntax.
- Both custom rules fired exactly once per matching request once bound to the correct interface, with no unrelated alerts interleaved — confirming the rules are specific enough not to generate noise from background traffic on the lab network.
- Because HOME_NET was left at its default value (Section 5.3), both rules correctly fired on traffic in either direction across eth0 — reinforcing why scoping HOME_NET precisely matters once a sensor moves from a single-host lab into a real network boundary.
- The ICMP alerts appeared within well under a second of each ping leaving its source, consistent with Snort's inline, packet-by-packet inspection model described in Section 2.3.

7. Results Summary

The table below maps each project requirement to the evidence captured during this exercise:

Evidence	Command / Source	Requirement Satisfied
Screenshots 1a–1c	hostnamectl, ip a, sudo apt update	Task 2 — Host & Network Configuration
Screenshot 2	snort -V (Snort++ 3.12.2.0 build flags)	Task 3 — IDS Installation Verification
Screenshots 3a–3e	snort ... -T across snort.conf / snort.lua / local.rules, plus a negative test	Task 3 — Configuration Syntax Validation
Screenshots 4a–4e	Live console — [SOC-ALERT] ICMP Ping Detected (loopback baseline + eth0, both directions)	Task 4 & 5 — ICMP Rule Detection
Screenshots 5a–5b	Live console — [SOC-ALERT] Inbound HTTP Connection	Task 4 & 5 — HTTP Web Rule Detection

Table 2: Evidence-to-requirement mapping for Week 9 submission.

8. Appendix A — Custom Rule File

For reference, the complete contents of /etc/snort/rules/local.rules as validated and tested in this exercise:

```
# Week 9 — SOC lab custom detection rules
# Author: Dwij Trivedi | DG Interns Hub Cyber Security Internship
# Rule 1: flag ICMP Echo Request (ping) traffic — basic reconnaissance detection
alert icmp any any -> $HOME_NET any (msg:"SOC-ALERT ICMP Ping Detected"; itype:8; sid:1000001; rev:1;)
# Rule 2: flag TCP connections on port 80 — basic plaintext HTTP visibility
```

```
alert tcp any any -> $HOME_NET 80 (msg:"SOC-ALERT Inbound HTTP Connection"; sid:1000002; rev:1;)
```

9. Remediation & Recommendations for Production Use

This lab intentionally keeps scope small — two rules, one sensor, one interface — in order to demonstrate the full install-to-alert workflow end to end. Moving this setup from a lab into a real SOC monitoring boundary would require the following hardening steps:

Gap Identified in This Lab	Recommended Production Fix
HOME_NET left at its default (any), so rules match traffic in either direction (Section 5.3).	Scope HOME_NET explicitly to the protected subnet(s), so only traffic actually inbound to protected assets is treated as reconnaissance, matching the intent behind each rule's message text.
Alerts print to an interactive console session tied to a logged-in terminal.	Run Snort as a background service (systemd unit) logging to unified2 or a syslog/JSON output that a SIEM can ingest, rather than a foreground console the analyst must keep open.
Only two narrowly-scoped custom rules are active, alongside no broader rule set tuning.	Layer the Snort Community/registered rule set on top of custom rules, then tune out noisy signatures specific to the environment before going live, as described in Section 2.1's MTTD/false-positive trade-off.
Detection is IDS-only (alert, never block).	Where business risk justifies it, promote selected highconfidence rules from IDS (alert) to IPS (drop) mode once false-positive rates have been proven low in alert-only operation first.

Table 3: Lab-to-production hardening recommendations.

10. Conclusion

This exercise moved from the conceptual (what a SOC is, how work is tiered across L1/L2/L3 analysts, and how SIEM, IDS and IPS divide responsibility across a defensive stack) to the concrete: a working Snort sensor that reliably detects both a reconnaissance technique (ICMP sweeps) and a basic connection-level event (HTTP traffic on port 80) using two short, purpose-built rules — including working through a realistic sequence of setup mistakes (the wrong legacy config path in Section 4.2, the wrong capture interface in Section 6.1) rather than a sanitized, first-try-perfect walkthrough.

The clearest takeaway is that an IDS's value is entirely a function of the quality and specificity of its rule set and its scoping (HOME_NET, interface selection) — the same engine that produced clean, specific alerts in this lab would be equally capable of flooding an analyst with false positives, or missing traffic entirely, if either were left misconfigured. This directly reinforces the SOC fundamentals covered in Task 1: tuning detection content and sensor placement to balance signal against noise is not a one-time setup step but the ongoing core work of L2/L3 analysts in any real SOC.

This lab also lays the groundwork for later weeks: the same sensor and local.rules file can be extended with additional signatures (e.g. for the IDOR and parameter-tampering traffic patterns identified against OWASP Juice Shop in the Week 8 report), and the alert output demonstrated here is the same console/log format a SIEM would ordinarily ingest, correlate and present to an L1 analyst for triage.